

```
In [ ]: import numpy as np
import pandas as pd
```

```
In [ ]: import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

```
In [10]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)
df = pd.read_csv("sales_data_sample.csv", encoding='latin1')
```

```
In [11]: df.head()
```

Out[11]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	STATUS	QTR_ID	MONTH_ID	YEAR_ID
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Shipped	1	2	2003
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Shipped	2	5	2003
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Shipped	3	7	2003
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Shipped	3	8	2003
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Shipped	4	10	2003

5 rows × 25 columns



In [12]:

```
df.info
```

```
Out[12]: <bound method DataFrame.info of
0      10107      30      95.70      2 2871.00
1      10121      34      81.35      5 2765.90
2      10134      41      94.74      2 3884.34
3      10145      45      83.26      6 3746.70
4      10159      49     100.00     14 5205.27
...      ...      ...      ...      ...      ...
2818     10350      20     100.00     15 2244.40
2819     10373      29     100.00      1 3978.51
2820     10386      43     100.00      4 5417.57
2821     10397      34      62.24      1 2116.16
2822     10414      47      65.52      9 3079.44
```

```
ORDERDATE  STATUS  QTR_ID  MONTH_ID  YEAR_ID  ... \
0  2/24/2003 0:00  Shipped      1         2     2003  ...
1  5/7/2003 0:00  Shipped      2         5     2003  ...
2  7/1/2003 0:00  Shipped      3         7     2003  ...
3  8/25/2003 0:00  Shipped      3         8     2003  ...
4 10/10/2003 0:00  Shipped      4        10     2003  ...
...      ...      ...      ...      ...      ...
2818 12/2/2004 0:00  Shipped      4        12     2004  ...
2819 1/31/2005 0:00  Shipped      1         1     2005  ...
2820 3/1/2005 0:00  Resolved      1         3     2005  ...
2821 3/28/2005 0:00  Shipped      1         3     2005  ...
2822 5/6/2005 0:00  On Hold      2         5     2005  ...
```

```
ADDRESSLINE1 ADDRESSLINE2 CITY STATE \
0      897 Long Airport Avenue      NaN      NYC      NY
1           59 rue de l'Abbaye      NaN      Reims      NaN
2  27 rue du Colonel Pierre Avia      NaN      Paris      NaN
3      78934 Hillside Dr.      NaN      Pasadena      CA
4      7734 Strong St.      NaN San Francisco      CA
...      ...      ...      ...      ...
2818      C/ Moralarzal, 86      NaN      Madrid      NaN
2819      Torikatu 38      NaN      Oulu      NaN
2820      C/ Moralarzal, 86      NaN      Madrid      NaN
2821      1 rue Alsace-Lorraine      NaN      Toulouse      NaN
2822      8616 Spinnaker Dr.      NaN      Boston      MA
```

```
POSTALCODE  COUNTRY  TERRITORY  CONTACTLASTNAME  CONTACTFIRSTNAME  DEALSIZE
```

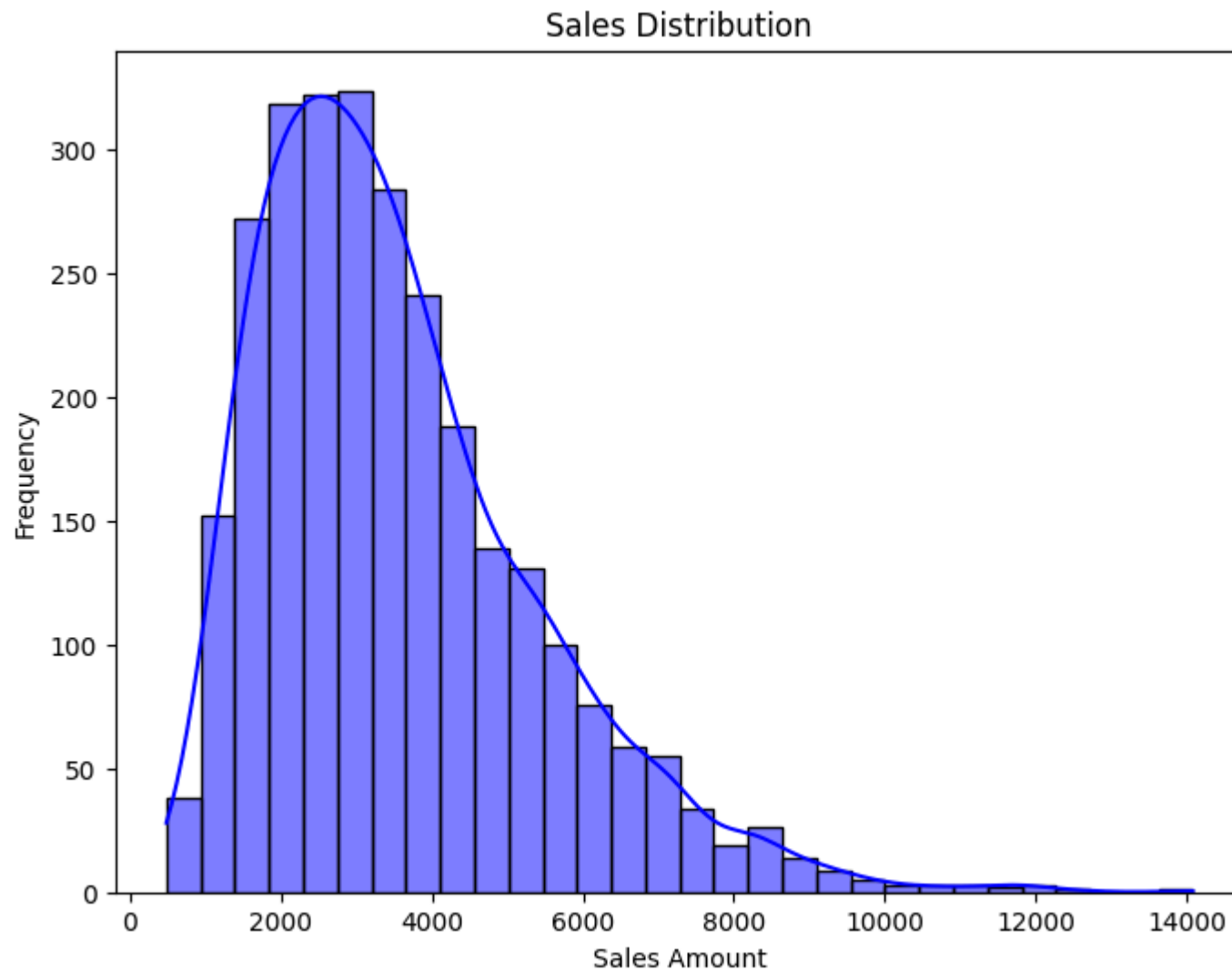
0	10022	USA	NaN	Yu	Kwai	Small
1	51100	France	EMEA	Henriot	Paul	Small
2	75508	France	EMEA	Da Cunha	Daniel	Medium
3	90003	USA	NaN	Young	Julie	Medium
4	NaN	USA	NaN	Brown	Julie	Medium
...	...	...	...	...	...	...
2818	28034	Spain	EMEA	Freyre	Diego	Small
2819	90110	Finland	EMEA	Koskitalo	Pirkko	Medium
2820	28034	Spain	EMEA	Freyre	Diego	Medium
2821	31000	France	EMEA	Roulet	Annette	Small
2822	51003	USA	NaN	Yoshido	Juri	Medium

[2823 rows x 25 columns]>

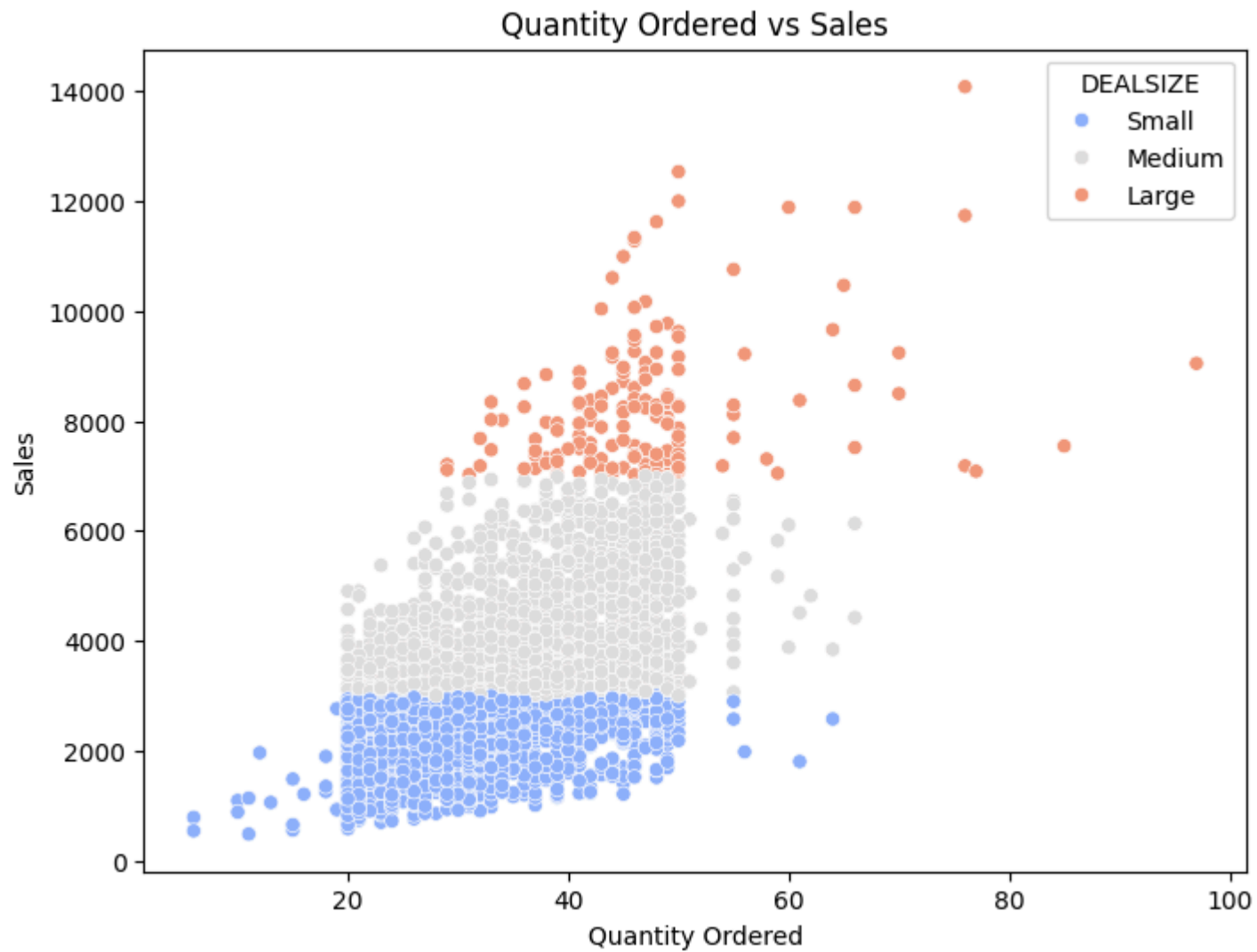
In [15]: `df.isnull().sum()`

```
Out[15]: ORDERNUMBER      0
          QUANTITYORDERED  0
          PRICEEACH        0
          ORDERLINENUMBER  0
          SALES             0
          ORDERDATE        0
          STATUS           0
          QTR_ID           0
          MONTH_ID         0
          YEAR_ID          0
          PRODUCTLINE      0
          MSRP             0
          PRODUCTCODE      0
          CUSTOMERNAME     0
          PHONE            0
          ADDRESSLINE1     0
          ADDRESSLINE2     2521
          CITY             0
          STATE            1486
          POSTALCODE       76
          COUNTRY          0
          TERRITORY        1074
          CONTACTLASTNAME  0
          CONTACTFIRSTNAME 0
          DEALSIZE         0
          dtype: int64
```

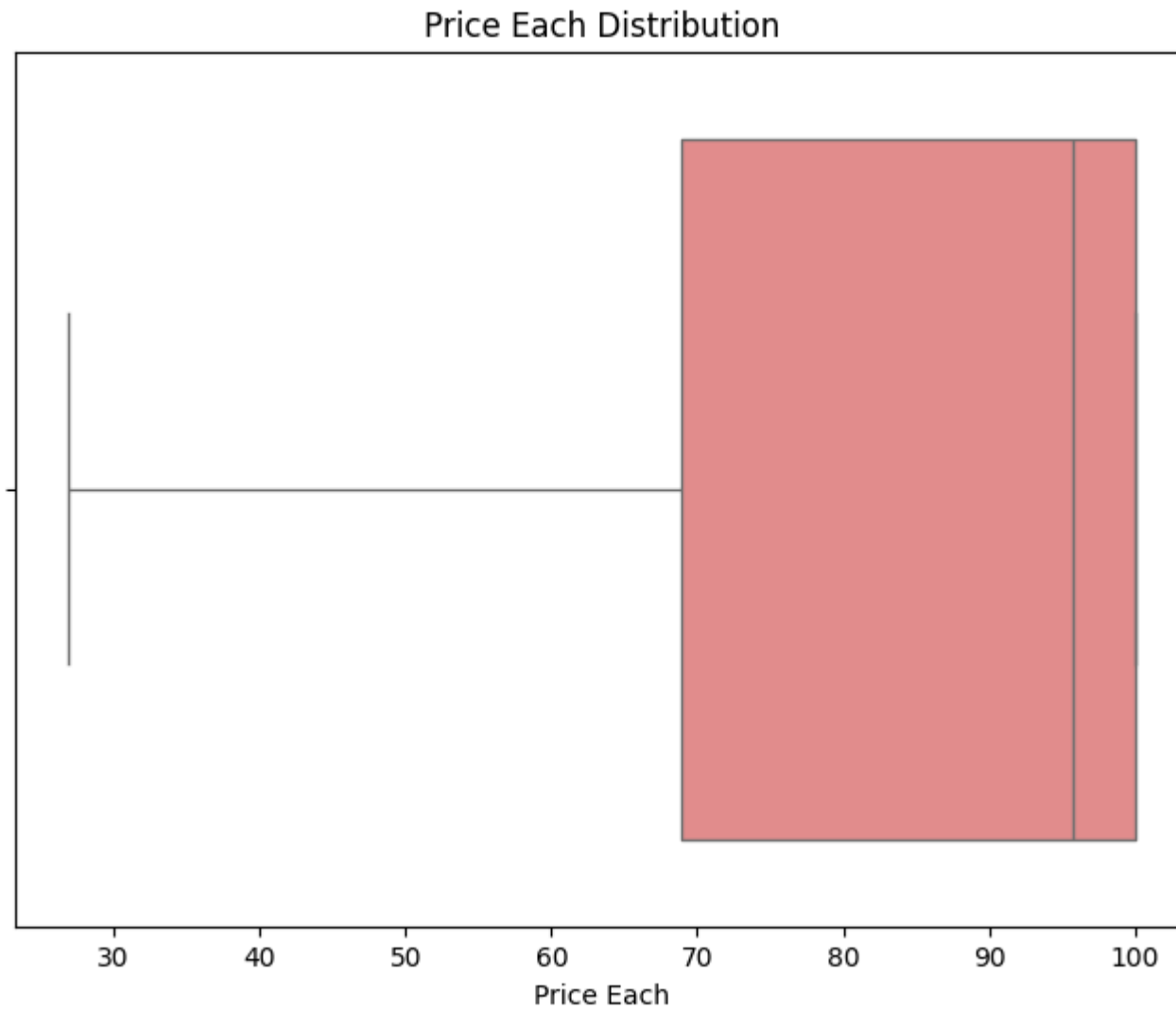
```
In [16]: plt.figure(figsize=(8,6))
          sns.histplot(df['SALES'], kde=True, color='blue', bins=30)
          plt.title('Sales Distribution')
          plt.xlabel('Sales Amount')
          plt.ylabel('Frequency')
          plt.show()
```



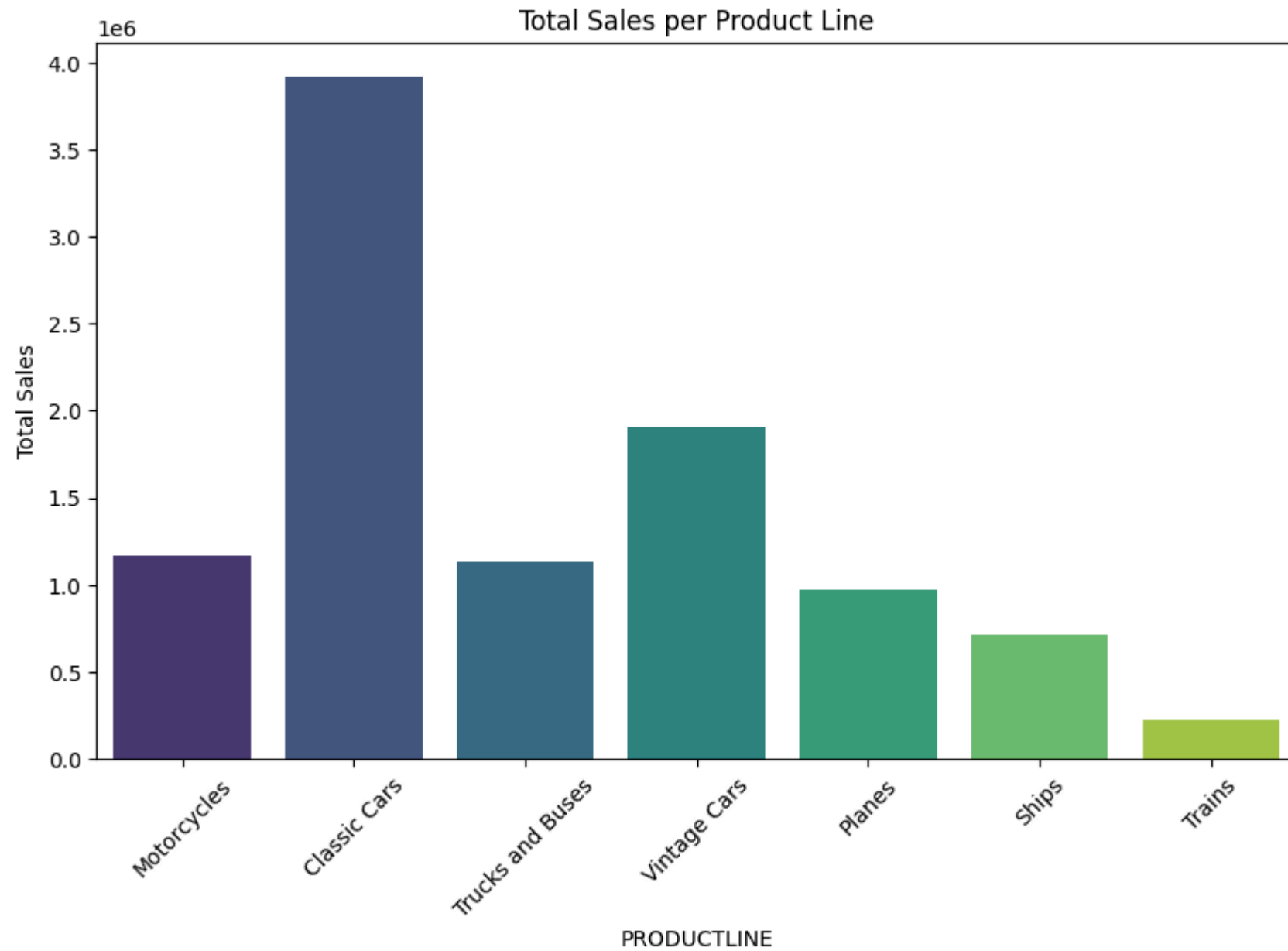
```
In [18]: plt.figure(figsize=(8,6))
sns.scatterplot(x='QUANTITYORDERED', y='SALES', data=df, hue='DEALSIZE', palette='coolwarm')
plt.title('Quantity Ordered vs Sales')
plt.xlabel('Quantity Ordered')
plt.ylabel('Sales')
plt.show()
```



```
In [19]: plt.figure(figsize=(8,6))
sns.boxplot(x='PRICEEACH', data=df, color='lightcoral')
plt.title('Price Each Distribution')
plt.xlabel('Price Each')
plt.show()
```

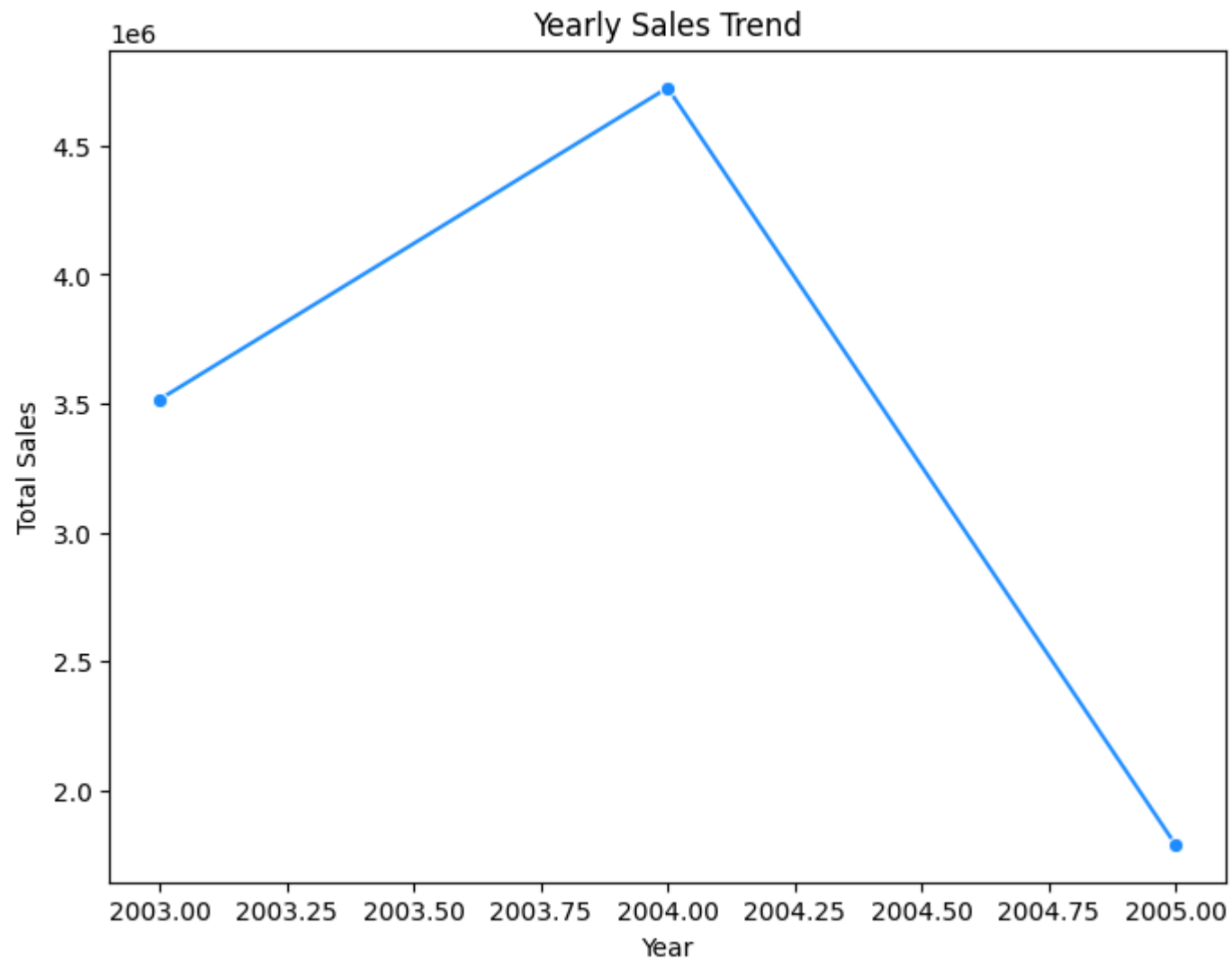


```
In [21]: plt.figure(figsize=(10,6))
sns.barplot(x='PRODUCTLINE', y='SALES', data=df, estimator=sum, ci=None, palette='viridis')
plt.title('Total Sales per Product Line')
plt.xticks(rotation=45)
plt.ylabel('Total Sales')
plt.show()
```



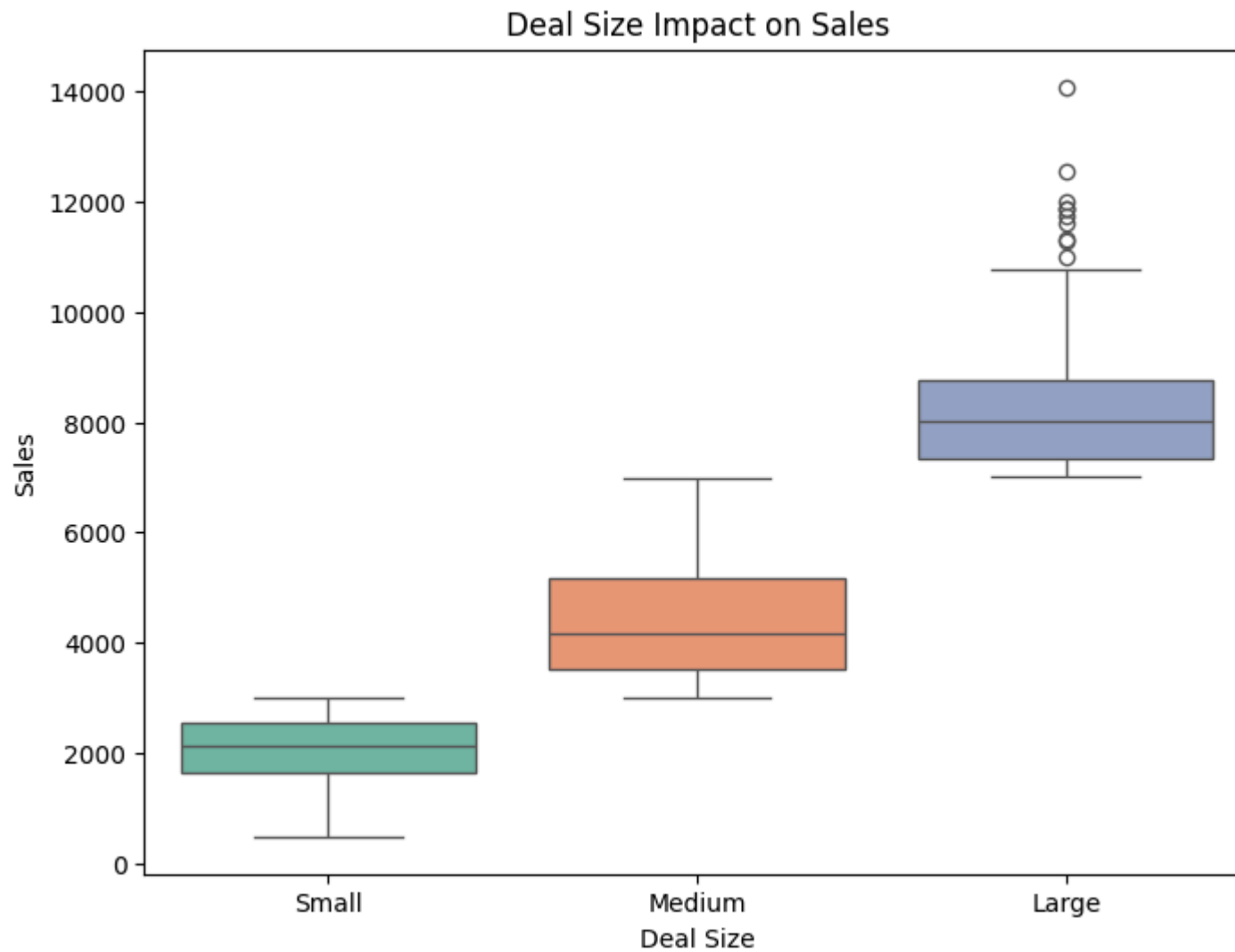
```
In [23]: plt.figure(figsize=(8,6))  
sales_per_year = df.groupby('YEAR_ID')['SALES'].sum().reset_index()
```

```
sns.lineplot(x='YEAR_ID', y='SALES', data=sales_per_year, marker='o', color='dodgerblue')  
plt.title('Yearly Sales Trend')  
plt.xlabel('Year')  
plt.ylabel('Total Sales')  
plt.show()
```



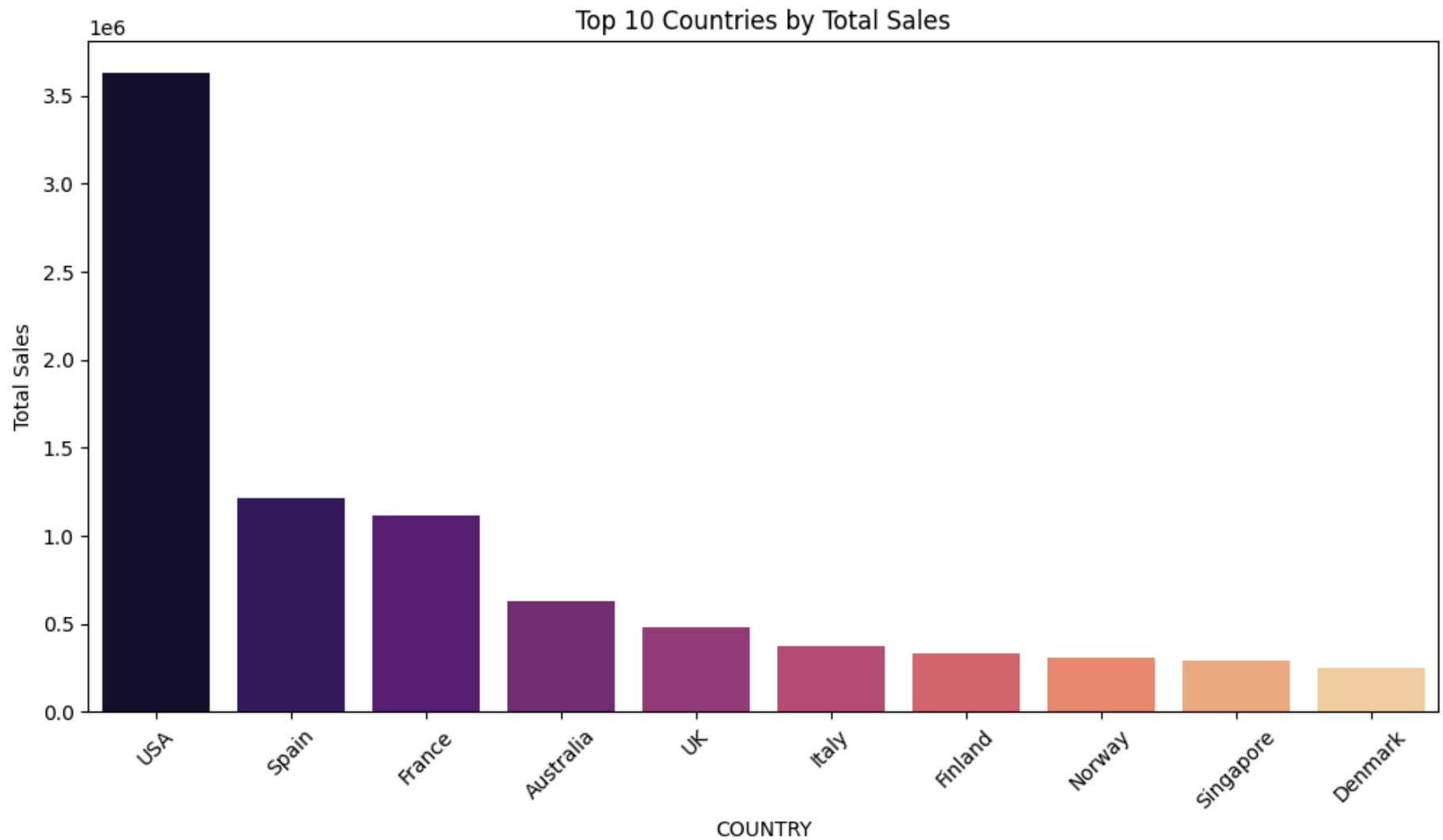
```
In [24]: plt.figure(figsize=(8,6))  
sns.boxplot(x='DEALSIZE', y='SALES', data=df, palette='Set2')
```

```
plt.title('Deal Size Impact on Sales')  
plt.xlabel('Deal Size')  
plt.ylabel('Sales')  
plt.show()
```



```
In [25]: plt.figure(figsize=(12,6))  
top_countries = df.groupby('COUNTRY')['SALES'].sum().nlargest(10).reset_index()  
sns.barplot(x='COUNTRY', y='SALES', data=top_countries, palette='magma')
```

```
plt.title('Top 10 Countries by Total Sales')  
plt.xticks(rotation=45)  
plt.ylabel('Total Sales')  
plt.show()
```

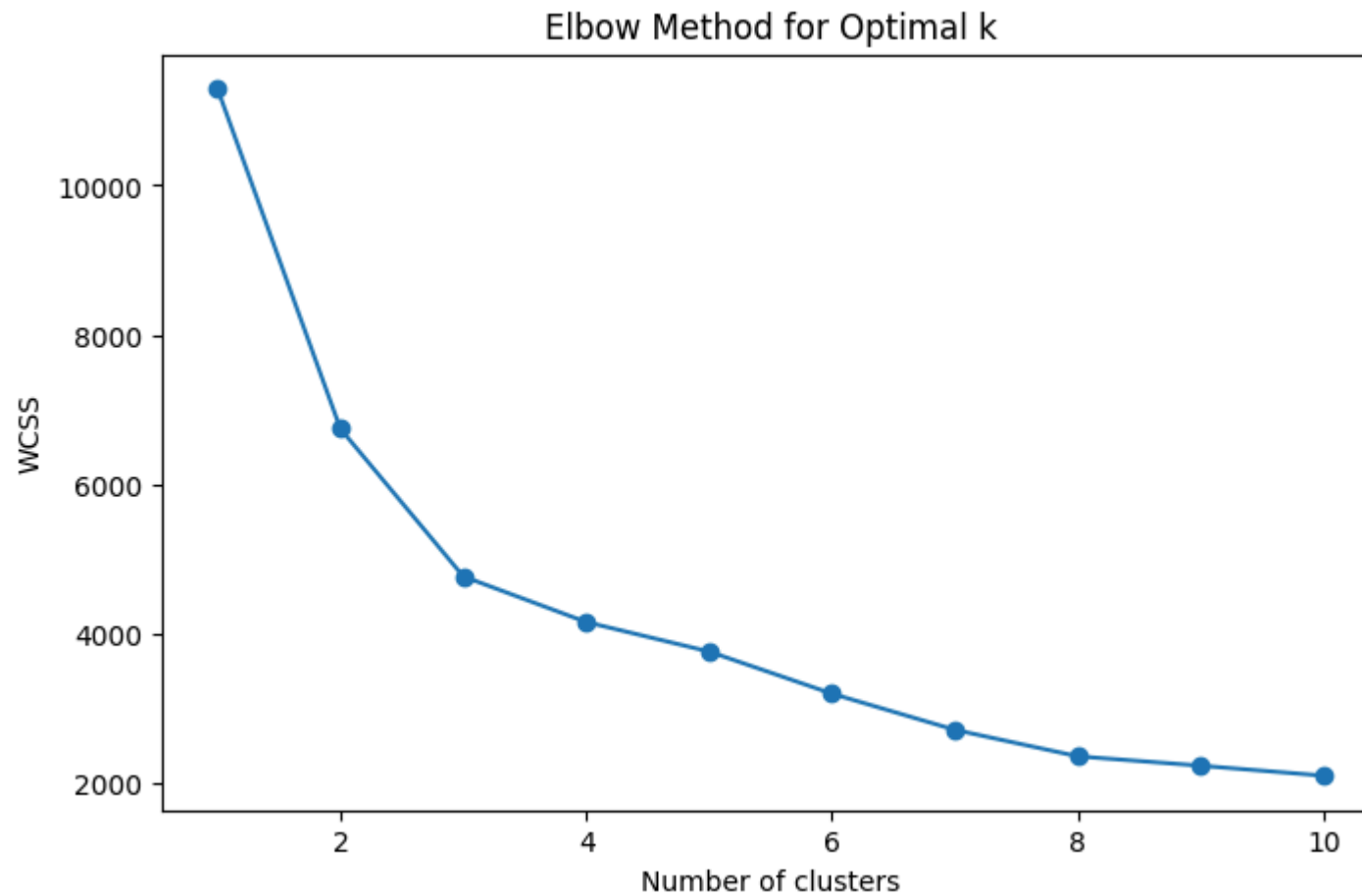


```
In [26]: features = ['QUANTITYORDERED', 'PRICEEACH', 'SALES', 'MSRP']  
X = df[features]
```

```
In [27]: # Handle missing values (if any)
X = X.dropna()
# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [30]: # Calculate WCSS for different cluster sizes
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot the Elbow Curve
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```



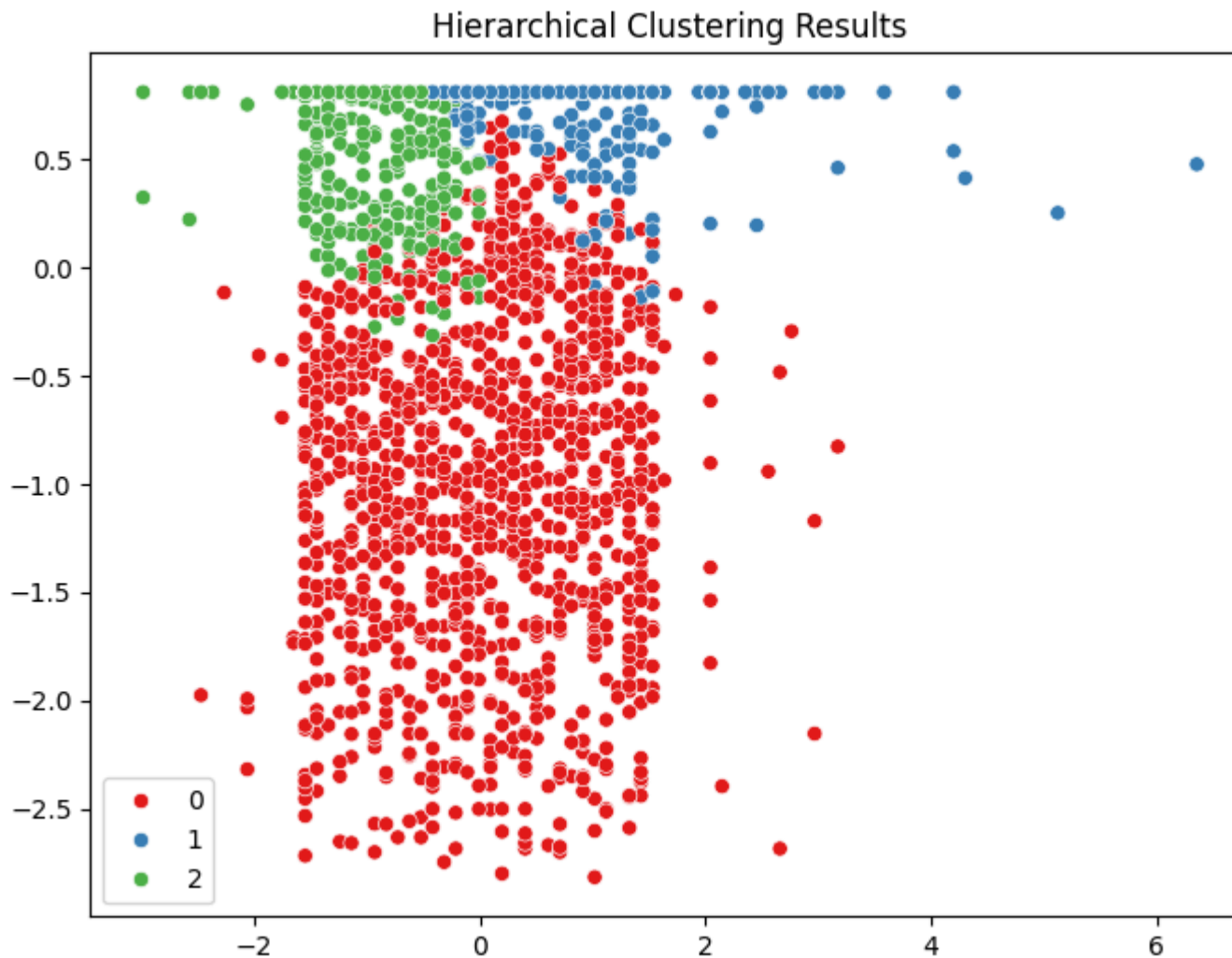
```
In [32]: # Choose k based on the elbow method, e.g., k=3
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
# Add the cluster labels to the original dataframe
df['Cluster'] = clusters
# Visualize the clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1], hue=clusters, palette='Set1')
plt.title('K-Means Clustering Results')
plt.show()
```



```
In [ ]: # Generate linkage matrix using the 'ward' method
linked = linkage(X_scaled, method='ward')
# Plot the dendrogram
plt.figure(figsize=(10, 7))
dendrogram(linked)
plt.title('Dendrogram for Hierarchical Clustering')
plt.xlabel('Samples')
```

```
plt.ylabel('Distance')  
plt.show()
```

```
In [40]: # Choose the number of clusters, e.g., 3  
hierarchical_clustering = AgglomerativeClustering(n_clusters=3, linkage='ward')  
hc_clusters = hierarchical_clustering.fit_predict(X_scaled)  
# Add the cluster labels to the original dataframe  
df['HC_Cluster'] = hc_clusters  
# Visualize the clusters  
plt.figure(figsize=(8, 6))  
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1], hue=hc_clusters, palette='Set1')  
plt.title('Hierarchical Clustering Results')  
plt.show()
```



```
In [42]: # Inertia for K-Means
inertia = kmeans.inertia_
print(f'Inertia: {inertia}')
from sklearn.metrics import silhouette_score
# Silhouette Score for K-Means
kmeans_silhouette = silhouette_score(X_scaled, clusters)
print(f'Silhouette Score (K-Means): {kmeans_silhouette}')
# Silhouette Score for Hierarchical Clustering
```

```
hc_silhouette = silhouette_score(X_scaled, hc_clusters)
print(f'Silhouette Score (Hierarchical): {hc_silhouette}')
from sklearn.metrics import davies_bouldin_score
# Davies-Bouldin Index for K-Means
kmeans_dbi = davies_bouldin_score(X_scaled, clusters)
print(f'Davies-Bouldin Index (K-Means): {kmeans_dbi}')
# Davies-Bouldin Index for Hierarchical Clustering
hc_dbi = davies_bouldin_score(X_scaled, hc_clusters)
print(f'Davies-Bouldin Index (Hierarchical): {hc_dbi}')
from sklearn.metrics import calinski_harabasz_score
# Calinski-Harabasz Index for K-Means
kmeans_ch = calinski_harabasz_score(X_scaled, clusters)
print(f'Calinski-Harabasz Index (K-Means): {kmeans_ch}')
# Calinski-Harabasz Index for Hierarchical Clustering
hc_ch = calinski_harabasz_score(X_scaled, hc_clusters)
print(f'Calinski-Harabasz Index (Hierarchical): {hc_ch}')
```

Inertia: 4766.021664461497

Silhouette Score (K-Means): 0.3508291135493167

Silhouette Score (Hierarchical): 0.31527494740967016

Davies-Bouldin Index (K-Means): 1.0090818209882544

Davies-Bouldin Index (Hierarchical): 0.9888199869306714

Calinski-Harabasz Index (K-Means): 1930.6762251406324

Calinski-Harabasz Index (Hierarchical): 1672.3553032169289

In []: